

对象序列化之三

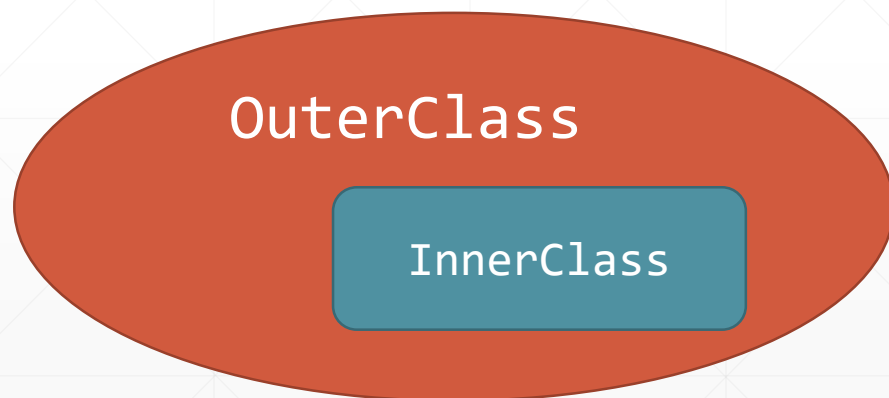


组合对象的序列化

北京理工大学计算机学院
金旭亮

序列化组合的对象

当两个对象间是组合的关系时，序列化最外部的对象，会“递归”地序列化其内部的对象。

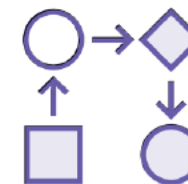


示例：ReadWriteEmbedObject.java

```
4 usages
class OuterClass implements Serializable {
    1 usage
    public OuterClass(InnerClass innerObject) {
        this.innerObject = innerObject;
    }
    2 usages
    public InnerClass innerObject = null;
    @Override
    public String toString() {
        return "OuterClass" + hashCode() + "{\n" +
            "\tinnerObject = " + innerObject +
            "\n}";
    }
}

3 usages
class InnerClass implements Serializable {
    @Override
    public String toString() {
        return "InnerClass" + hashCode();
    }
}
```

序列化与反序列化组合对象



```
1 usage
private static void writeEmbedObj() throws IOException {
    try (ObjectOutputStream out = new ObjectOutputStream(
        new FileOutputStream(objDataFileName))) {
        //创建一个包容有内部对象的组合对象
        OuterClass container = new OuterClass(new InnerClass());
        //序列化组合对象
        out.writeObject(container);
        System.out.println("复合对象已经写到文件" + objDataFileName + "中。");
    }
}
```

```
1 usage
private static void readEmbedObj()
    throws IOException, ClassNotFoundException {
    System.out.println("从" + objDataFileName + "文件中读取复合对象。");
    try (ObjectInputStream in = new ObjectInputStream(
        new FileInputStream(objDataFileName))) {
        //反序列化外部对象
        OuterClass other = (OuterClass) in.readObject();
        //输出外部对象的字段值，以证明其确实顺利地重新创建了。
        System.out.println(other);
    }
}
```

注意，要使用这种“嵌套序列化”技术特性，要求参与的对象必须都实现了Serializable接口。

```
Run: ReadWriteEmbedObject x
"C:\Program Files\Java\jdk-17\bin\java.exe"
复合对象已经写到文件embedObj.dat中。
从embedObj.dat文件中读取复合对象。
OuterClass1789447862{
    innerObject = InnerClass1642534850
}
Process finished with exit code 0
```

共享对象的序列化



如果有多个外部对象组合了“同一个”对象，那么，序列化这些外部对象时，会不会导致这个被组合的“内部”对象被序列化多次？

要序列化的组合对象

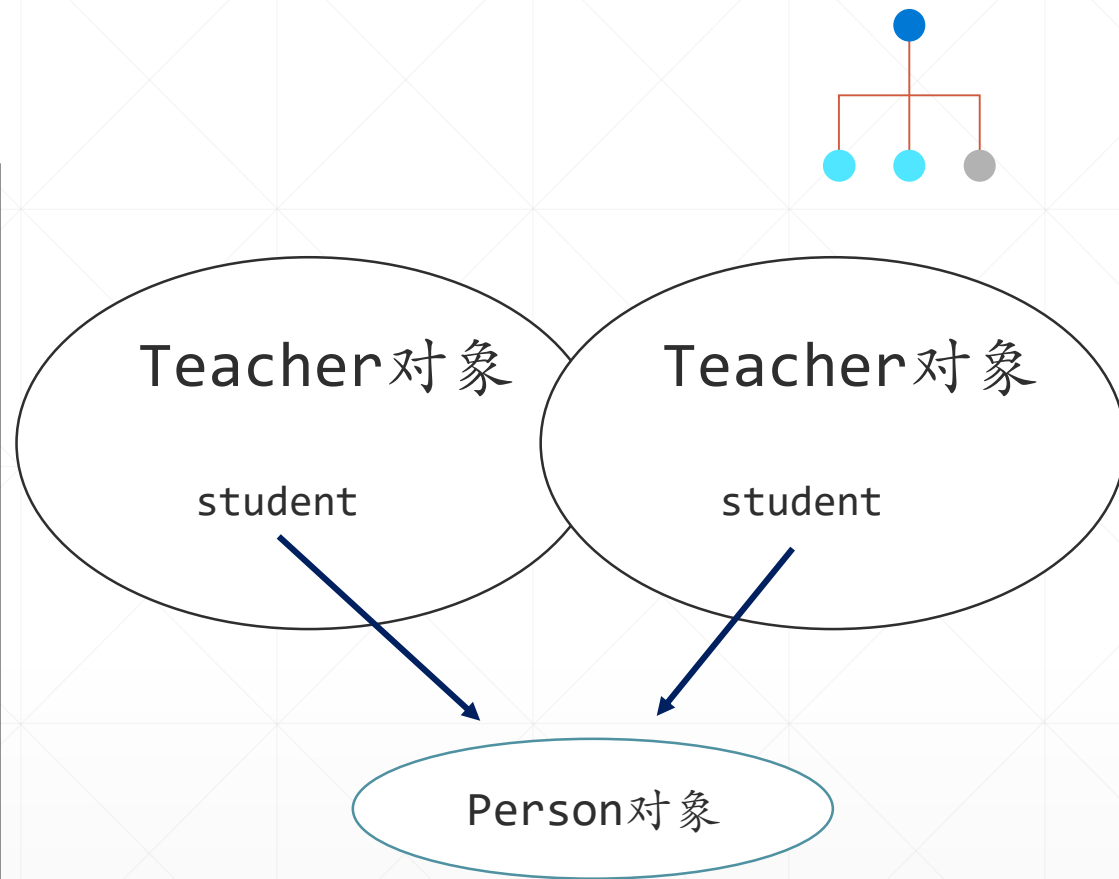
10 usages

```
public class Teacher
    implements java.io.Serializable {
    4 usages
    private String name;
    4 usages
    private Person student;

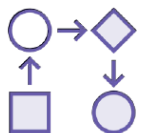
    2 usages
    public Teacher(String name, Person student) {...}

    @Override
    public String toString() {
        return name + "的学生是" + student.getName();
    }

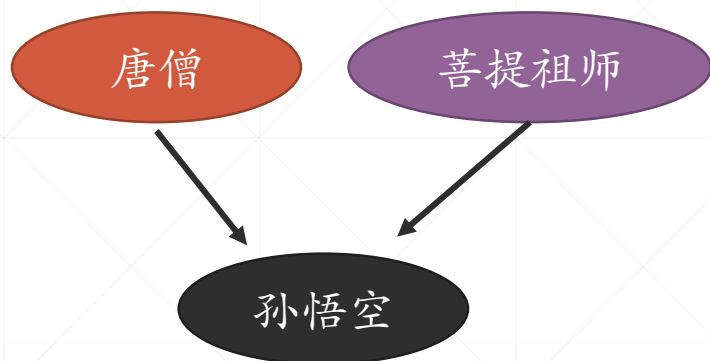
    getter and setter
}
```



一个学生，可以有多个老师，也就是说，多个老师对象，它的student字段，可以引用同一个学生对象。



序列化过程



```
Run: ReadWriteTeacher x
"C:\Program Files\Java\jdk-17\bin\
以下对象已被写入到teacher.dat中
(1)唐僧的学生是孙悟空
(2)菩提祖师的学生是孙悟空
(3)孙悟空有500岁
(4)菩提祖师的学生是孙悟空
Process finished with exit code 0
```

```
private static void writeToFile() throws IOException {
    try (ObjectOutputStream oos = new ObjectOutputStream(
        new FileOutputStream(dataFileName))) {
        //创建一个学生，两个老师对象，注意，两个老师对象共享同一个学生对象
        Person student = new Person(name: "孙悟空", age: 500);
        Teacher teacher1 = new Teacher(name: "唐僧", student);
        Teacher teacher2 = new Teacher(name: "菩提祖师", student);
        //依次将四个对象写入输出流
        oos.writeObject(teacher1);
        oos.writeObject(teacher2);
        oos.writeObject(student);
        oos.writeObject(teacher2); // "菩提祖师"对象被序列化了两次
        System.out.println("以下对象已被写入到" + dataFileName + "中");
        System.out.println("(1)" + teacher1);
        System.out.println("(2)" + teacher2);
        System.out.println("(3)" + student);
        System.out.println("(4)" + teacher2);
    }
}
```

ReadWriteTeacher.java

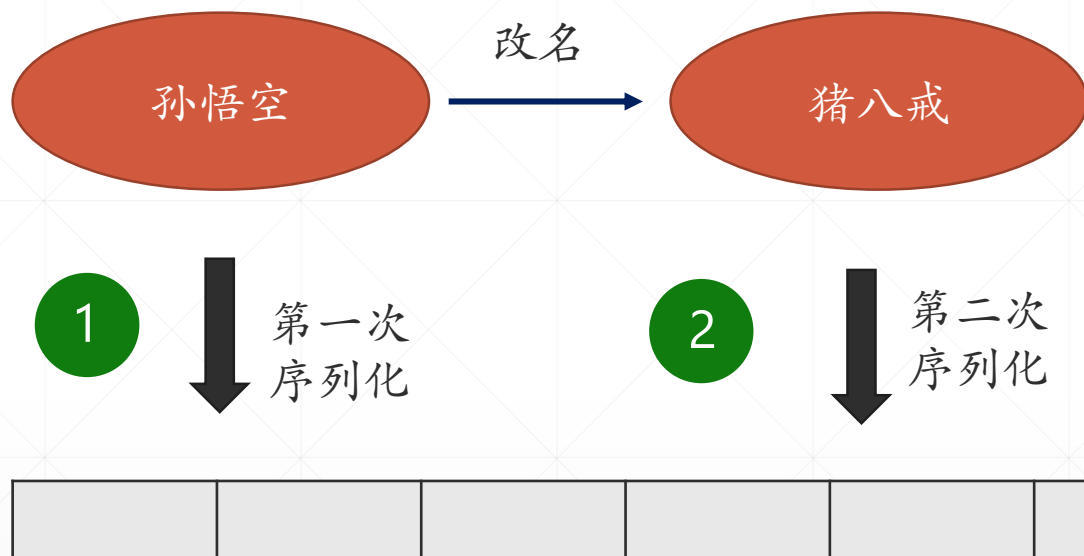
反序列化过程

```
private static void readFromFile()
    throws IOException, ClassNotFoundException {
    try (ObjectInputStream ois = new ObjectInputStream(
        new FileInputStream(dataFileName))) {
        //依次读取ObjectInputStream输入流中的四个对象
        Teacher teacher1 = (Teacher) ois.readObject();
        System.out.println("从" + dataFileName + "中读出对象teacher1: " + teacher1);
        Teacher teacher2 = (Teacher) ois.readObject();
        System.out.println("从" + dataFileName + "中读出对象teacher2: " + teacher2);
        Person student = (Person) ois.readObject();
        System.out.println("从" + dataFileName + "中读出对象student: " + student);
        Teacher teacher3 = (Teacher) ois.readObject();
        System.out.println("从" + dataFileName + "中读出对象teacher3: " + teacher3);
        System.out.println("唐僧的student字段, 是否引用孙悟空对象? "
            + (teacher1.getStudent() == student)); //输出true
        System.out.println("菩提祖师对象一的student字段, 是否引用孙悟空对象? "
            + (teacher2.getStudent() == student)); //输出true
        System.out.println("菩提祖师对象二的student字段, 是否引用孙悟空对象? "
            + (teacher3.getStudent() == student)); //输出true
        System.out.println("菩提祖师对象一和菩提祖师对象二, 是否是同一个对象? "
            + (teacher1 == teacher3)); //输出false
    }
}
```

对于被多个对象所共享的共享对象（本例中为student对象），序列化时，它只是在第一次序列化时将数据写入流中，后继序列化，Java只是向流中写入一个标识，不会反复将相同的数据复制多次，之后反序列化时，此共享对象只被反序列化一次，之后，让组合它的其他对象相应字段“引用”这个唯一的共享对象。

```
ReadWriteTeacher x
"C:\Program Files\Java\jdk-17\bin\java.exe" "-java
从teacher.dat中读出对象teacher1: 唐僧的学生是孙悟空
从teacher.dat中读出对象teacher2: 菩提祖师的学生是孙悟空
从teacher.dat中读出对象student: 孙悟空有500岁
从teacher.dat中读出对象teacher3: 菩提祖师的学生是孙悟空
唐僧的student字段, 是否引用孙悟空对象? true
菩提祖师对象一的student字段, 是否引用孙悟空对象? true
菩提祖师对象二的student字段, 是否引用孙悟空对象? true
菩提祖师对象一和菩提祖师对象二, 是否是同一个对象? false
```

共享对象的状态更改



前面示例中被共享的学生对象，如果在序列化后状态有更改，会带来非常麻烦的问题，因为组合它的对象在序列化时，往往会需要多次序列化共享对象，这就可能造成数据更改丢失的问题。

数据文件：mutable.dat

先将“孙悟空”对象序列化到文件中，将后，将同一对象改名“猪八戒”，再次序列化到文件中。


```

public static void main(String[] args) {
    try (var oos = new ObjectOutputStream(new FileOutputStream(dataFileName));
        var ois = new ObjectInputStream(new FileInputStream(dataFileName))) {
        Person student = new Person( name: "孙悟空", age: 500);
        //序列化学生对象, 此时, 其name属性为"孙悟空"
        oos.writeObject(student);
        System.out.println("对象:" + student + "已写入文件" + dataFileName);
        //改变Person对象的name属性
        student.setName("猪八戒");
        //再次序列化学生对象
        oos.writeObject(student);
        System.out.println("对象:" + student + "已写入文件" + dataFileName);
        //验证序列化结果
        Person student1 = (Person) ois.readObject();
        System.out.println("从文件" + dataFileName + "中读出对象:" + student1);
        Person student2 = (Person) ois.readObject();
        System.out.println("从文件" + dataFileName + "中读出对象:" + student2);
        //反序列化后, 两个学生变量, 是否引用同一个学生对象?
        System.out.println(student1 == student2); //输出: true
        //学生的姓名是哪个?
        System.out.println(student2.getName()); //输出: 孙悟空
    } catch (Exception ex) {
        ex.printStackTrace();
    }
}

```

SerializeMutable.java



由于Java不会重复序列化同一个对象, 因此, 如果在两次序列化中间, 此对象的属性值被“临时”更改了, 注意, 这一更改不会被保存, 因此, 反序列化时, 得到的是“第一次”被序列化时刻的值。

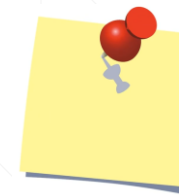
```

Run: SerializeMutable x
"C:\Program Files\Java\jdk-17\bin\java.exe"
对象:"孙悟空有500岁"已写入文件mutable.dat
对象:"猪八戒有500岁"已写入文件mutable.dat
从文件mutable.dat中读出对象:孙悟空有500岁
从文件mutable.dat中读出对象:孙悟空有500岁
true
孙悟空

Process finished with exit code 0

```

开发建议



用于共享的对象，最好是只读的。



不要把同一个对象序列化两次以上。